

第四次小班课个人资料

Optimal Merging Tree

最佳归并树



1

定义与背景

2

算法思想

3

存储实现

4

具体构建方法

5

性质与特点

6

应用场景

目录



一. 定义与背景

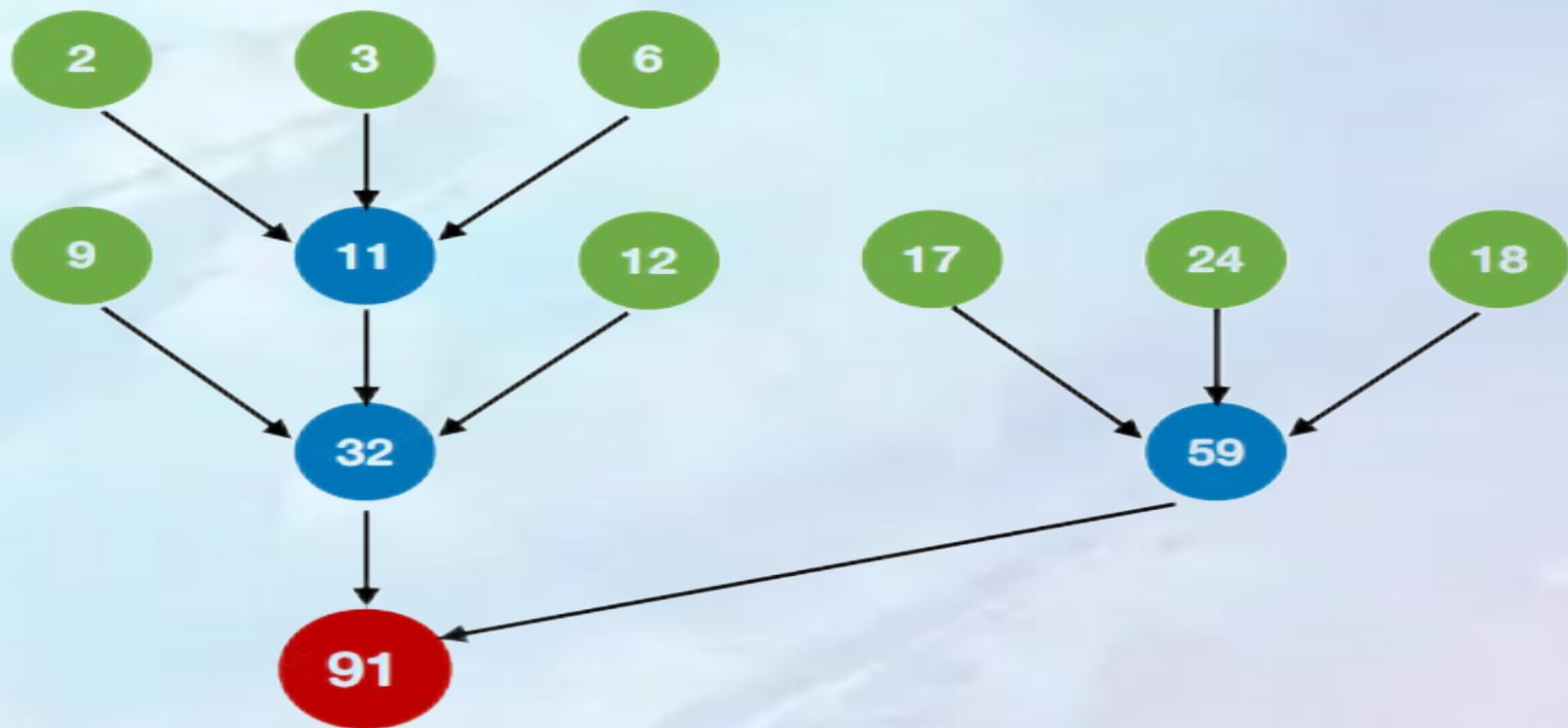
外部排序是指处理无法全部加载到内存中的大数据集时，利用磁盘等外部存储设备进行的排序。由于磁盘I/O操作是外部排序的主要瓶颈，因此减少I/O次数是提高排序效率的关键。最佳归并树便是在这种背景下被提出的一种优化策略。^[1]



一. 定义与背景

- 最佳归并树的概念

最佳归并树是一种特殊的树形结构，用于组织归并段，以便在归并过程中最小化磁盘I/O次数。它类似于哈夫曼树（Huffman Tree），在构建过程中会考虑归并段的长度（或占用的块数量），以使得整个归并过程的总I/O次数最小。



二. 算法思想

最佳归并树 (Optimal Merge Tree) 是一种优化外部排序中归并过程的策略，旨在最小化对外部存储的读写次数。其核心思想是将初始归并段的长度视为权重，构建一棵带权路径长度 (WPL) 最小的 k 叉哈夫曼树。在归并过程中，每个记录需读写一次，因此总I/O次数与归并树的WPL成正比。当初始归并段数不足以构成严格的 k 叉树时，可引入权重为0的“虚段”以满足构造条件。通过这种方式，最佳归并树有效降低了外部排序中的I/O成本，提高了整体效率。

三. 存储实现

最佳归并树 (Optimal Merge Tree) 在外部排序中用于优化归并过程，以最小化磁盘I/O操作次数。其存储实现主要依赖于以下两种结构：

- **k叉哈夫曼树 (k-ary Huffman Tree) :** 将初始归并段的长度视为权重，构建一棵带权路径长度 (WPL) 最小的k叉树。该树的构建方式类似于哈夫曼编码，通过每次选取权重最小的k个节点合并，直到形成一棵完整的树。为了满足k叉树的结构要求，若初始归并段数不足以构成严格的k叉树，可引入权重为0的“虚段”以补足节点数。这种结构确保了归并过程中总的I/O操作次数最少。

三. 存储实现

- 败者树 (Loser Tree)：在实际归并操作中，败者树是一种高效的选择结构。它是一棵完全二叉树，叶子节点代表各个归并段的当前最小元素，非叶子节点存储在比较中失败的元素索引。通过这种结构，可以在每次归并操作中快速找到当前最小的元素，并在元素更新后仅需对相关路径进行调整，优化了比较次数和内存操作。在实现中，败者树通常使用数组来表示，以便于在内存中高效操作。每个归并段对应一个输入缓冲区，归并结果通过输出缓冲区写回外存。这种存储方式不仅减少了磁盘I/O次数，还提高了归并操作的整体效率。
- 亦可利用动态规划方法，系统求解最优二叉查找树的结构和最小期望代价。动态规划的基本思想是：将最优二叉树问题分解为规模更小的子问题，通过递归公式组合子问题的最优解。[2]

三. 存储实现

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int optimalMergeCost(vector<int>& files) {
    // 小根堆
    priority_queue<int, vector<int>, greater<int>> minHeap(files.begin(), files.end());
    int totalCost = 0;

    while (minHeap.size() > 1) {
        int first = minHeap.top();
        minHeap.pop();
        int second = minHeap.top();
        minHeap.pop();

        int cost = first + second;
        totalCost += cost;
        minHeap.push(cost);
    }
    return totalCost;
}
```



假设有 n 个文件（或节点），每次合并操作都要在堆（优先队列）中插入和弹出元素,时间复杂度为 $O(n\log n)$

四. 具体构建方法

构建最佳归并树的简要方法如下：

- 初始化节点：将每个初始归并段视为一个叶子节点，其权重为该段的记录数。
- 确定k值：根据可用的归并路数（k）选择构建k叉树。
- 补充虚段（如需）：若初始段数不足以构成严格的k叉树，计算需添加的权重为0的“虚段”数量，公式为：

若 $(m - 1) \bmod (k - 1) \neq 0$ ，则需添加 $(k - 1) - ((m - 1) \bmod (k - 1))$ 个虚段。

- 构建归并树：使用类似哈夫曼树的贪心策略，反复选择权重最小的k个节点合并为新节点，直到形成一棵完整的k叉树。

通过上述步骤，构建的最佳归并树可最小化归并过程中的磁盘I/O操作次数，提高外部排序的效率。

四. 具体构建方法

例如，现有通过置换选择排序算法所得到的 9 个初始归并段，其长度分别为：9，30，12，18，3，17，2，6，24。在对其采用 3-路平衡归并的方式时可能出现如图 1 所示的情况：

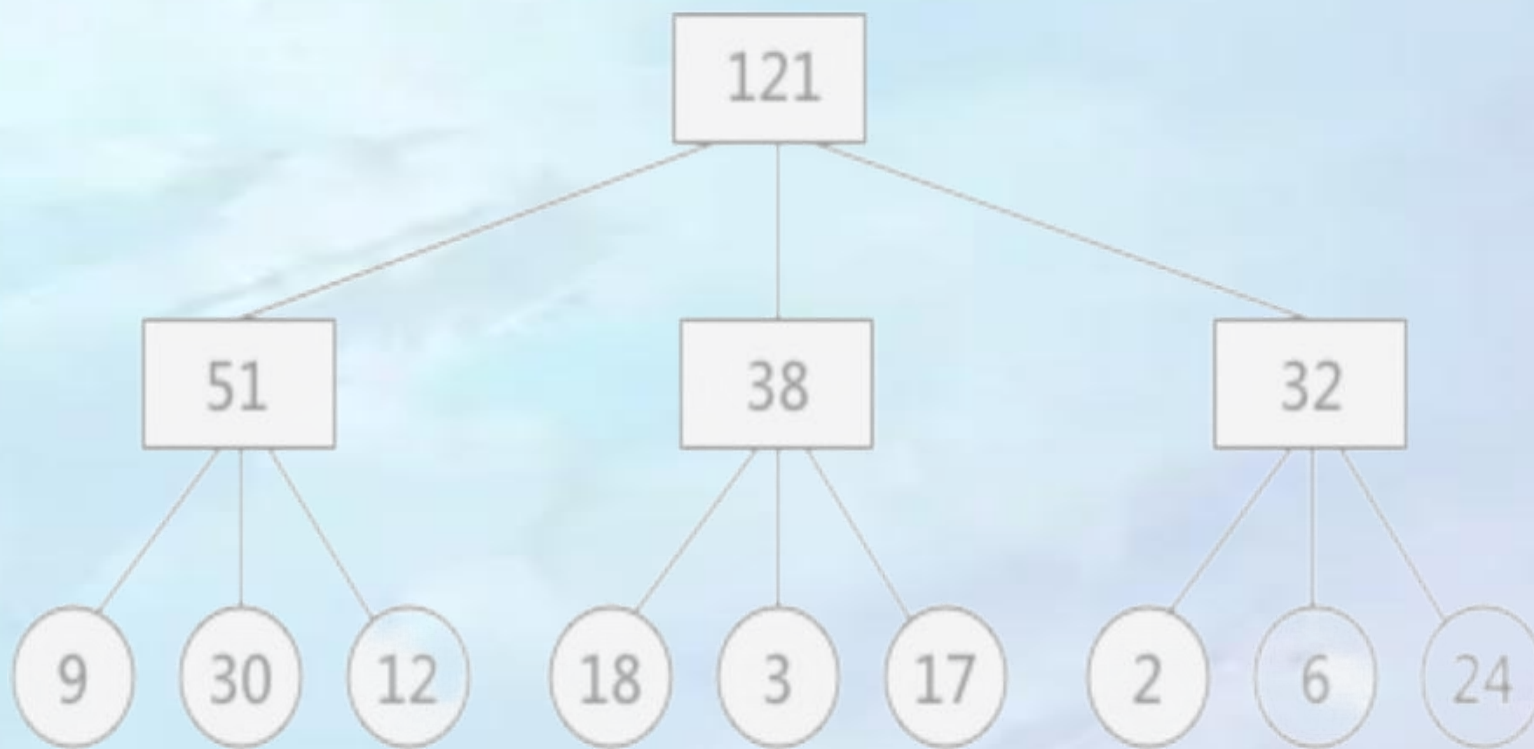


图 1 3-路平衡归并

四. 具体构建方法

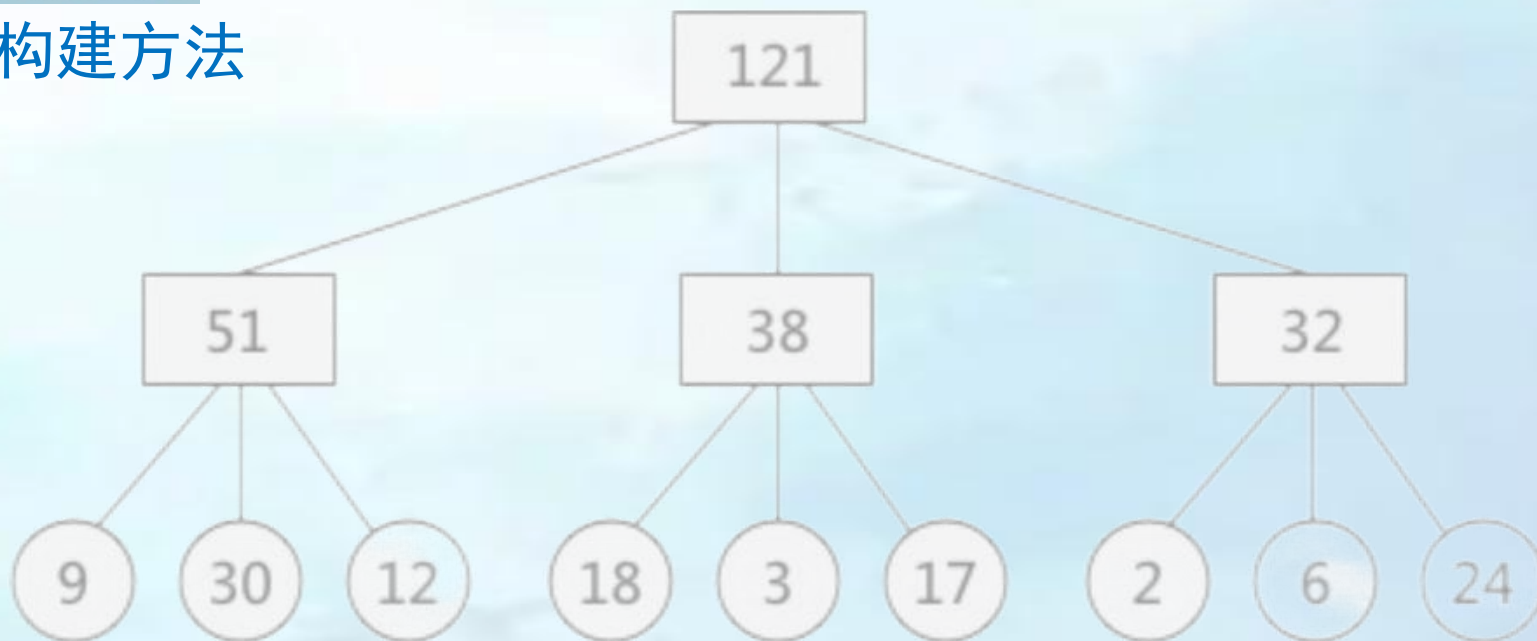


图 1 3-路平衡归并

假设在进行平衡归并时，操作每个记录都需要单独进行一次对外存的读写，那么图 1 中的归并过程需要对外存进行读或者写的次数为：

$(9+30+12+18+3+17+2+6+24) * 2 * 2 = 484$ （图 1 中涉及到了两次归并，对外存的读和写各进行 2 次）

从计算结果上看，对于图 1 中的 3 叉树来讲，其操作外存的次数恰好是树的带权路径长度的 2 倍。所以，对于如何减少访问外存的次数的问题，就等同于考虑如何使 k-路归并所构成的 k 叉树的带权路径长度最短。

四. 具体构建方法

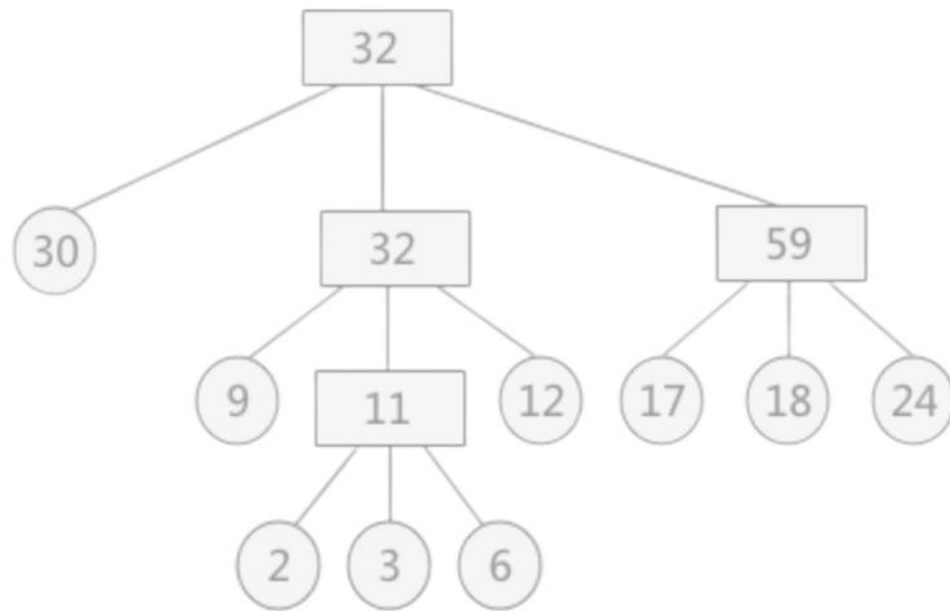


图 2 赫夫曼树作为3-路归并树

若想使树的带权路径长度最短，就是构造哈夫曼树

其对外存的读写次数为：

$$(2*3+3*3+6*3+9*2+12*2+17*2+18*2+24*2+30)*2=446$$

通过以构建赫夫曼树的方式构建归并树，使其对读写外存的次数降至最低

（k-路平衡归并，需要选取合适的 k 值，构建哈夫曼树作为归并树）。所以称此归并树为最佳归并树。^[3]

四. 具体构建方法

若在一般情况下，对于 k -路平衡归并来说，若 $(m-1) \bmod (k-1) = 0$ ，则不需要增加虚段；否则需附加 $k - (m-1) \bmod (k-1) - 1$ 个虚段。

9 个初始归并段改为 8 个，在做 3-路平衡归并的时候就需要有一个结点的度为 2。虚段的设置只是为了方便构建赫夫曼树，在构建完成后虚段自动去掉即可。

对于具体设置哪个结点的度为 2，为了使总的带权路径长度最短，正确的选择方法是：附加一个权值为 0 的结点（称为“虚段”），然后再构建赫夫曼树。例如图 2 中若去掉权值为 30 的结点，其附加虚段的最佳归并树如图 3 所示：

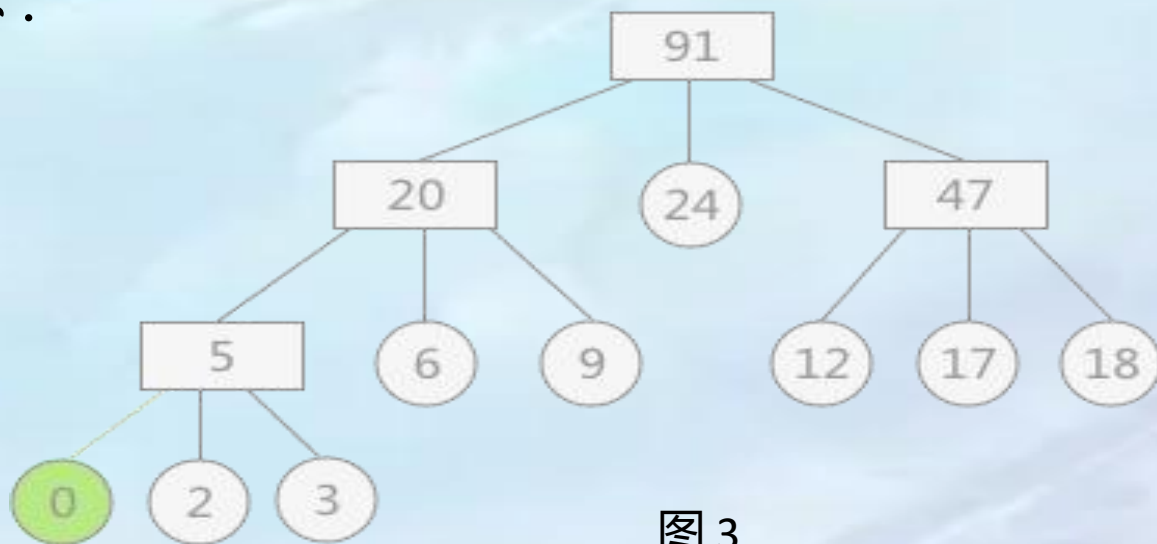


图 3

五. 性质与特点

最小化I/O次数:

最佳归并树通过优化归并段的组织方式,使得整个归并过程的总I/O次数最小。

灵活性:

最佳归并树可以适应不同数量的初始归并段和不同的归并路数(k 值),具有较强的灵活性。

高效性:

在构建最佳归并树的过程中,可以利用一些高效的算法和数据结构(如败者树等)来加速归并过程。

五. 性质与特点

理论基础

每个初始归并段对应一个叶子结点，把归并段的块数作为叶子的权值

归并树的WPL=树中所有叶结点的带权路径长度之和

归并过程中的磁盘I/O次数=归并树的WPL*2

六. 应用场景

最佳归并树在外部排序中具有广泛的应用。例如，在大数据处理、数据库管理系统和文件系统等领域中，经常需要处理无法全部加载到内存中的大数据集。此时，可以利用最佳归并树来进行高效的外部排序。

在实际应用中，可以根据具体的数据集和硬件条件来选择合适的归并路数（ k 值）和初始归并段的长度，以构建最佳归并树并优化排序性能。

最佳归并树的本质是解决多路合并的最优顺序问题，其核心思想是每次都合并当前最小的两项。典型应用包括外部排序、哈夫曼编码、数据压缩、任务调度等需要最小化合并成本的场景。

- [1] Hopcroft, J.E., & Ullman, J.D. (1971). Optimal Merging Algorithms. SIAM Journal on Computing, 2(4), 294-303.
- [2] Knuth, D. E. (1971). Optimal binary search trees. Acta Informatica, 1, 14–25.
- [3] https://blog.csdn.net/qq_52442214/article/details/

Thanks!

